MLOps: machine learning in production

Run models like any other production system: data checks, reproducible training, tested pipelines, a model registry, safe releases, serving, and monitoring for drift with a retraining loop.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What MLOps is and how ML systems fail (14 min)
2. Data and feature pipelines for ML (15 min)
3. Experiment tracking and reproducibility (13 min)
4. Training pipelines and testing models in CI (15 min)
5. Model registry, packaging and safe releases (14 min)
6. Serving models: batch, online and streaming (15 min)
7. Monitoring drift and retraining (15 min)
8. Debugging ML systems in production (16 min)
9. Feature stores and data versioning (12 min)
10. Model evaluation, A/B tests and shadow deployments (13 min)
11. ML pipelines and workflow orchestration (12 min)
12. LLMOps: versioning prompts, models and evaluations (12 min)
Cheat sheet

Lesson 1. What MLOps is and how ML systems fail

14 minutes. A model is only a small part of a machine learning system. This lesson shows the parts around it and the quiet ways they fail in production.

A normal service fails loudly: a crash, an error rate, a timeout. A machine learning system can fail while every dashboard is green. The server answers, the latency is fine, and the predictions are slowly getting worse. MLOps is the set of habits that make those quiet failures visible and fixable, by applying the same ideas you already use for software: versioning, testing, automated delivery and monitoring.

What you will learn

- Name the parts of a production ML system beyond the model
- Explain why a model can be wrong while the service looks healthy
- Define training-serving skew and silent failure
- Say what MLOps adds to DevOps and what it does not replace

The model is the small box

Code for the model itself is usually a small part of the system. Around it sit data collection, data checks, feature computation, training, evaluation, packaging, serving, monitoring and the process to retrain. Most production trouble lives in those surrounding parts. If you only version the model file, you have versioned the smallest piece.

Two things change, not one

In ordinary software the code changes and the behaviour follows. In ML the code, the data and the model all change, and the behaviour depends on all three. The same code with new data gives a different model. A model that never changed can degrade because the world changed under it. So you must track data and model versions as carefully as code.

Training-serving skew

Skew means the model sees different inputs in production than it saw in training. The cause is often boring: a feature computed in a notebook with one library and in the service with another, a unit that changed, a column that is now empty, or a time zone. The model is not broken. It is being fed something it never learned from.

Silent failure

A fraud model that starts approving more risky payments raises no error. A recommendation model that favours old items still returns a list. You only see the damage through a business metric or a quality check, often days later. This is why monitoring in MLOps includes the data and the predictions, not only CPU and latency.

What MLOps adds

Add four things to your DevOps toolbox: data and model versioning, automated evaluation as a release gate, monitoring of inputs and outputs, and a retraining path that is as safe as a code deploy. You still need containers, pipelines, observability and on-call.

MARGA by EknathaLabs
marga.eknathalabs.com

What a healthy service can hide

```
$ curl -s localhost:8080/health
{"status":"ok","model":"churn-v14"}
$ curl -s localhost:8080/metrics | grep -E "latency|errors"
predict_latency_seconds_p95 0.041
predict_errors_total 0
$ python check_inputs.py --window 24h
feature days_since_login: 31% null (training: 0.2% null)
feature plan_tier: 4 new values not seen in training
```

The service is healthy by every operational signal, yet a third of one feature is missing. Only a data check shows the problem. The output is a simulated illustration.

How it fails

A feature pipeline changes upstream and fills a column with nulls. A model trained on last year stays in production for two years. Training used a clean sample while production sees messy data. Nobody owns the question "is the model still good?", so nobody asks it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How What MLOps is and how ML systems fail fits together

Part	What it does	What goes wrong
The model is the small box	Code for the model itself is usually a small part of the system.	Versioning the model file but not the data or features.
Two things change, not one	In ordinary software the code changes and the behaviour follows.	Treating a deployed model as finished.
Training-serving skew	Skew means the model sees different inputs in production than it saw in training.	Computing features twice, once for training and once for serving.
Silent failure	A fraud model that starts approving more risky payments raises no error.	Monitoring only latency and errors.
What MLOps adds	Add four things to your DevOps toolbox: data and model versioning, automated evaluation as a release gate, monitoring of inputs and outputs, and a retraining path that...	Check the Common mistakes list below this diagram for what usually goes wrong here.
What a healthy service can hide	The service is healthy by every operational signal, yet a third of one feature is missing.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: The model is the small box to Two things change, not one / Two things change, not one to Training-serving skew / Training-serving skew to Silent failure / Silent failure to What MLOps adds / What MLOps adds to What a healthy service can hide

Incident: A green dashboard and a falling conversion rate

> **Incident:** *A team shipped a ranking model and watched latency and errors. Both stayed healthy. Weeks later the business noticed a drop in conversions. An upstream change had renamed a field, so one important feature arrived empty and the model quietly ranked worse.*

```
$ python check_inputs.py --window 7d
feature item_popularity: 100% null since 2026-03-02
feature price_bucket: ok
$ git log --oneline -1 upstream-schema
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. The lesson is real and common: input checks on every feature, with an alert when a feature goes null or shifts, would have caught this on day one instead of week three.

MARGA by EknathaLabs marga.eknathalabs.com

Interview question

"Our model has 99.9 percent uptime but results got worse. What do you check?" Look at input data first (nulls, ranges, new categories), then compare live feature distributions with training, then check upstream changes, then evaluate the model on recent labelled data.

Common mistakes

- Monitoring only latency and errors
- Versioning the model file but not the data or features
- Computing features twice, once for training and once for serving
- Treating a deployed model as finished

Check yourself

Why can an ML service be wrong while all operational dashboards are green?

Answer: Predictions can degrade from data or world changes without causing errors or latency

What is training-serving skew?

Answer: The model sees different inputs in production than it saw in training

Which three things change in an ML system?

Answer: Code, data and the model

What is a good first protection against silent failure?

Answer: Checks and alerts on input features and prediction distributions

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Data and feature pipelines for ML

15 minutes. Models are only as reliable as the data feeding them. Learn data validation, versioning and keeping training and serving features the same.

Most machine learning incidents start in the data, not in the model. A feature pipeline is a production system like any other, with inputs that change, jobs that fail and schemas that drift. This lesson covers how to check data before it is used, how to version it so a result can be reproduced, and how to compute features once so training and serving agree.

What you will learn

- Validate data against a schema and expectations before training
- Version datasets so a model can be traced to its data
- Explain what a feature store is for and when you do not need one
- Keep training and serving features computed by the same code

Treat data like code and like an API

Data has a shape: columns, types, ranges and meanings. Write that shape down as a schema and test every batch against it. A column that is suddenly 40 percent null, a value outside a plausible range, or a new category should stop the pipeline or raise an alert, just as a failing unit test stops a build.

Useful checks

Start with a short list: row count within an expected band, no unexpected nulls, values within range, categories from a known set, no duplicate keys, freshness (the newest record is recent), and a distribution check against a reference. These catch most real problems and run in seconds.

Version the data

If you cannot say which data trained a model, you cannot reproduce it or explain a bad result. Store datasets as immutable snapshots with an id, or use a tool that records content hashes next to your code commit. Never overwrite a training set in place.

Point-in-time correctness

When you build training rows, each feature must use only information that existed at the time of the label. Using today's value to predict yesterday's outcome leaks the future and makes the model look great in testing and fail in production. Join features as of the event time.

One definition of a feature

Write each feature once and use the same code for the training set and for serving. A feature store is a managed way to do that: it keeps an offline copy for training and an online copy for low-latency lookups, with one definition. Small teams can start with a shared library and add a store later.

A data check you can run in a pipeline

```
# checks.yaml: expectations for the daily training table
table: churn_training
checks:
  - row_count: {min: 90000, max: 130000}
  - freshness: {column: event_date, max_age_hours: 30}
  - not_null: [customer_id, plan_tier, event_date]
  - unique: [customer_id, event_date]
  - range: {column: days_since_login, min: 0, max: 3650}
  - allowed_values: {column: plan_tier, values: [free, basic, pro, team]}
  - null_rate: {column: last_payment_amount, max: 0.05}
on_fail: stop_pipeline_and_page
```

Each line is a test on data, evaluated before training starts. A failing check stops the pipeline instead of training on bad rows.

MARGA by EknathaLabs marga.eknathalabs.com

How it fails

A schema changes upstream and the pipeline keeps running. A join silently drops half the rows. A feature uses information from after the event. The training set is edited by hand and nobody recorded how. A fix is applied to serving but not to training.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Data and feature pipelines for ML fits together

Part	What it does	What goes wrong
Treat data like code and like an API	Data has a shape: columns, types, ranges and meanings.	Training on data that was never validated.
Useful checks	Start with a short list: row count within an expected band, no unexpected nulls, values within range, categories from a known set, no duplicate keys, freshness (the...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Version the data	If you cannot say which data trained a model, you cannot reproduce it or explain a bad result.	Overwriting a dataset in place.
Point-in-time correctness	When you build training rows, each feature must use only information that existed at the time of the label.	Using future information in a training feature.
One definition of a feature	Write each feature once and use the same code for the training set and for serving.	Writing features twice, once for training and once for serving.
A data check you can run in a	Each line is a test on data, evaluated before training starts.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Treat data like code and like an API to Useful checks / Useful checks to Version the data / Version the data to Point-in-time correctness / Point-in-time correctness to One definition of a feature / One definition of a feature to A data check you can run in a

Incident: The feature that knew the future

> **Incident:** *A team built a model to predict customer cancellations and reported excellent test results. In production it performed far worse. One feature, "support tickets this month", included tickets raised after the customer had already decided to leave, which the model could not know at prediction time.*

```
$ python audit_features.py --model churn-v9
feature support_tickets_month: uses data up to month end
label cancel_date: can precede month end
leakage risk: HIGH
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration of a common mistake called leakage. Building training rows with point-in-time joins, so a feature only uses data from before the prediction moment, removes this class of problem.

Interview question

"How do you stop bad data reaching a model?" Define a schema and expectations, run them on every batch before training and before serving, version the datasets, and make the pipeline fail closed with an alert instead of continuing with bad rows.

Common mistakes

- Training on data that was never validated
- Overwriting a dataset in place
- Using future information in a training feature

MARGA by EknathaLabs marga.eknathalabs.com

- Writing features twice, once for training and once for serving

Check yourself

What should happen when a data check fails in a training pipeline?

Answer: The pipeline stops and alerts instead of training on bad data

Why version datasets?

Answer: So a model can be traced to the exact data that trained it and reproduced

What is point-in-time correctness?

Answer: A feature uses only information that existed at the moment of the prediction

What does a feature store mainly help with?

Answer: Defining features once and using them for both training and low-latency serving

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Experiment tracking and reproducibility

13 minutes. Record what produced each model so you can compare runs, repeat a result and explain a decision later.

Training a model is closer to running experiments than to building a feature. You try settings, compare numbers and keep the best. Without records, the best model becomes a mystery: nobody remembers which data, code or parameters produced it. Experiment tracking turns that mess into a searchable history, and reproducibility means you can run the same recipe again and get the same result.

What you will learn

- List what to record for every training run
- Explain why runs should be compared on the same data and metric
- Control randomness so a result can be repeated
- Link a model in production back to its run

What to record

For every run, record the code version, the data version, the parameters, the environment (library versions, hardware), the random seed, the metrics, and the output files. Together these are the recipe. If any one is missing, you can look at a number but you cannot trust or repeat it.

A tracking server

Tools such as MLflow, or any system that stores runs, give you a place to log parameters and metrics from code and then compare runs in a table or chart. The tool matters less than the habit: every training job writes a run record automatically, with no manual notes.

Compare fairly

A new model beating an old one means little if they used different data splits or metrics. Fix the evaluation set and the metric, keep it out of training, and compare on the same slice. Report the metric on important sub-groups too, because an average can hide a group the model serves badly.

Control randomness

Random seeds, data shuffling and some GPU operations make results vary between runs. Set seeds, pin library versions and record hardware. Some noise cannot be removed, so run important comparisons several times and look at the spread. A one-point gain inside the noise is not a gain.

Reproduce on demand

Package training in a container with pinned dependencies, run it from a commit, and read data by version id. Then "retrain model 14 exactly" becomes a command, not an archaeology project. This also makes audits and rollbacks safe.

Logging a run

```
$ mlflow run . -P learning_rate=0.05 -P max_depth=6 --env-manager=local
run_id: 7c1e9a40
code_version: a94f3c2
data_version: churn-2026-03-01
seed: 42
metric auc_valid: 0.871
metric auc_valid_new_customers: 0.804
artifact: model/ (12 MB)
$ mlflow runs compare 7c1e9a40 3b88d216
```

One command trains and records code, data, seed and metrics together. The new-customer slice is logged too, so an average does not hide a weak group. The output is a simulated illustration.

MARGA by EknathaLabs
marga.eknathalabs.com

How it fails

Results live in a notebook that was changed after the run. The data path points at a file that was overwritten. A seed is unset, so nobody can repeat the number. Two people compare runs on different test sets and argue about it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Experiment tracking and reproducibility fits together

Part	What it does	What goes wrong
What to record	For every run, record the code version, the data version, the parameters, the environment (library versions, hardware), the random seed, the metrics, and the output...	Leaving seeds and library versions unrecorded.
A tracking server	Tools such as MLflow, or any system that stores runs, give you a place to log parameters and metrics from code and then compare runs in a table or chart.	Keeping results in notebooks with no run record.
Compare fairly	A new model beating an old one means little if they used different data splits or metrics.	Comparing runs on different test sets.
Control randomness	Random seeds, data shuffling and some GPU operations make results vary between runs.	Choosing a model on a difference smaller than the noise.
Reproduce on demand	Package training in a container with pinned dependencies, run it from a commit, and read data by version id.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Logging a run	One command trains and records code, data, seed and metrics together.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What to record to A tracking server / A tracking server to Compare fairly / Compare fairly to Control randomness / Control randomness to Reproduce on demand / Reproduce on demand to Logging a run

Incident: The best model nobody could rebuild

> **Incident:** *A team deployed the best model from weeks of experiments. When it needed retraining, the new run was much worse. The notebook had been edited, the training file replaced and the library upgraded, and nobody had recorded the original settings.*

```
$ python train.py --config best.yaml
auc_valid: 0.842
$ python compare.py --old model-prod --new run-latest
code version: unknown for model-prod
data version: unknown for model-prod
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration of a very common situation. Recording code, data, parameters and environment for every run would have made the old result repeatable and the gap explainable.

Interview question

"How do you make an ML result reproducible?" Version code and data, pin the environment in a container, fix seeds, log parameters and metrics to a tracking system, and evaluate on a fixed held-out set.

Common mistakes

- Keeping results in notebooks with no run record
- Comparing runs on different test sets
- Leaving seeds and library versions unrecorded
- Choosing a model on a difference smaller than the noise

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Which is NOT needed to reproduce a training run?

Answer: The name of the person who started it

Why compare runs on the same evaluation set and metric?

Answer: Otherwise a difference may come from the test, not the model

What does a tracking server mainly provide?

Answer: A searchable history of runs with parameters, metrics and artifacts

What should you do when two models differ by a tiny amount?

Answer: Repeat runs and check whether the difference is bigger than the noise

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Training pipelines and testing models in CI

15 minutes. Turn training into an automated, tested pipeline, with checks on data, code and model quality before anything is released.

A model trained by hand from a laptop cannot be released safely or repeated reliably. A training pipeline runs the same steps every time: fetch data, validate, build features, train, evaluate and package. Continuous integration then adds tests, so a change to code or data cannot ship a worse model without somebody seeing it first.

What you will learn

- Describe the stages of a training pipeline
- Write tests for data, code and model quality
- Use an evaluation gate to block a worse model
- Run training jobs in containers with the right resources

The stages

A typical pipeline has these stages: ingest data, validate it, build features, train, evaluate, register the model and notify. Each stage reads versioned inputs and writes versioned outputs, so a failed stage can be rerun without redoing everything and any result can be traced.

Three kinds of tests

Test the code with ordinary unit tests, for example that feature functions give expected results on small inputs. Test the data with expectations on every batch. Test the model: it must load, accept the expected input shape, return outputs in a valid range and meet a minimum quality bar on a fixed evaluation set.

The evaluation gate

Before a model is registered, compare it with the current production model on the same held-out data. Block the release if the key metric is worse by more than a small margin, or if any important group gets worse. A gate turns "I think it is better" into a measured rule.

Behavioural checks

Add simple behaviour tests: a tiny change that should not matter does not flip the answer, an obviously bad input is handled, and known hard cases still work. They are cheap, and they catch failures that a single average hides.

Containers and resources

Run each stage in a container with pinned dependencies and explicit CPU, memory and GPU requests. Expensive GPU nodes should run only the training step, while cheap nodes run validation and packaging. Add timeouts and retries so one stuck job does not block the queue.

MARGA by EknathaLabs
marga.eknathalabs.com

A pipeline with an evaluation gate

```
name: train-churn-model
on:
  schedule: [{cron: "0 2 * * 1"}]
  push: {paths: ["features/**", "train/**"]}
jobs:
  validate-data:
    run: python -m checks.run --config checks.yaml
  train:
    needs: validate-data
    resources: {gpu: 1, memory: 16Gi}
    run: python train/train.py --data-version $DATA_VERSION --seed 42
  evaluate:
    needs: train
    run: >
      python eval/gate.py --candidate $RUN_ID --baseline prod
      --metric auc --min-delta -0.002 --groups new_customers,enterprise
  register:
    needs: evaluate
    run: python registry/register.py --run $RUN_ID --stage staging
```

Each job depends on the one before it. The gate allows a tiny drop in the overall metric but blocks any release that makes a named group worse, so a bad model never reaches the registry.

How it fails

A pipeline trains on unvalidated data. A gate that only looks at the average lets a model that fails one customer group through. Training and serving use different library versions. A failed job is retried forever and fills the GPU queue.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Training pipelines and testing models in CI fits together

Part	What it does	What goes wrong
The stages	A typical pipeline has these stages: ingest data, validate it, build features, train, evaluate, register the model and notify.	Retrying failed GPU jobs without limits.
Three kinds of tests	Test the code with ordinary unit tests, for example that feature functions give expected results on small inputs.	Check the Common mistakes list below this diagram for what usually goes wrong here.
The evaluation gate	Before a model is registered, compare it with the current production model on the same held-out data.	Evaluating only on the average metric.
Behavioural checks	Add simple behaviour tests: a tiny change that should not matter does not flip the answer, an obviously bad input is handled, and known hard cases still work.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Containers and resources	Run each stage in a container with pinned dependencies and explicit CPU, memory and GPU requests.	Training and serving in different environments.
A pipeline with an evaluation gate	Each job depends on the one before it. The gate allows a tiny drop in the overall metric but blocks any release that makes a named group worse, so a bad model never...	No gate between training and the registry.

Flow: The stages to Three kinds of tests / Three kinds of tests to The evaluation gate / The evaluation gate to Behavioural checks / Behavioural checks to Containers and resources / Containers and resources to A pipeline with an evaluation gate

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: The retrain that shipped a worse model

> **Incident:** *A weekly retraining job replaced the production model automatically. One week, a data problem lowered quality for new customers. The average metric barely moved, so nothing flagged it, and the support team noticed first.*

```
$ python eval/gate.py --candidate run-88 --baseline prod --metric auc
overall auc: 0.866 (prod 0.869)  within tolerance
new_customers auc: 0.741 (prod 0.803)  BELOW LIMIT
result: FAIL, not registered
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Checking named groups in the gate, not only the average, turns a quiet regression into a failed build before it reaches users.

Interview question

"How would you automate model releases safely?" Build a pipeline with data checks, tests, a fixed evaluation set, a comparison against the production model that includes important groups, and an approval or canary step before full release.

Common mistakes

- Evaluating only on the average metric
- Training and serving in different environments
- No gate between training and the registry
- Retrying failed GPU jobs without limits

Check yourself

What does an evaluation gate do?

Answer: Blocks a candidate model that is worse than the baseline on agreed metrics

Which test catches a feature function bug?

Answer: A unit test on small known inputs

Why include groups in the gate?

Answer: An average can hide a group the model now serves badly

Why pin dependencies in training containers?

Answer: So the same code and versions give repeatable results

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Model registry, packaging and safe releases

14 minutes. Store models with their history, package them for serving, and release with shadow tests, canaries and fast rollback.

Once a model passes its checks, it needs a home and a safe path to users. A model registry gives each version a name, a lineage and a stage. Packaging makes the model run the same way everywhere. A careful release then exposes the model to real traffic in small steps, with a quick way back.

What you will learn

- Describe what a model registry stores and why
- Package a model with its dependencies and interface
- Compare shadow, canary and A/B release methods
- Roll back a model in minutes

What a registry holds

A registry stores each model version with its run id, code and data versions, metrics, owner, and a stage such as staging, production or archived. It answers "what is live, who approved it, what trained it and what was the previous one?" in one place.

Package it like a service

Wrap the model in a container with pinned libraries, a clear input schema and a health endpoint. Include the exact feature code or call it from a shared library. Record the image digest in the registry, so the thing you tested is the thing you run.

Shadow release

In shadow mode the new model receives a copy of real traffic, but its answers are only logged, not used. You compare it with the live model on real inputs with zero user risk. It also shows latency and error rates under real load.

Canary and A/B

A canary sends a small share of traffic, say 1 to 5 percent, to the new model and watches errors, latency and quality metrics before widening. An A/B test goes further and measures a business outcome between groups, which needs enough traffic and time to be meaningful.

Rollback

Keep the previous model loaded or one command away. Make rollback a registry stage change, not a rebuild. Decide the triggers in advance: error rate, latency, a quality proxy or a business metric beyond a limit. Practise it before you need it.

Moving a model through stages

```
$ mlreg show churn --stage production
version: 13  image: registry.example.com/churn@sha256:4f2a...
trained: run 3b88d216  data: churn-2026-02-01
$ mlreg promote churn 14 --to staging
$ deploy canary churn --version 14 --share 5%
canary: error_rate 0.1% latency_p95 44ms (baseline 41ms)
$ mlreg promote churn 14 --to production --keep-previous 13
$ mlreg rollback churn --to 13   # if a trigger fires
```

The registry holds the version, the image digest and the previous version. A canary at five percent is checked before promotion. The output is a simulated illustration.

MARGA by EknathaLabs marga.eknathalabs.com

How it fails

The model file is copied by hand to a server. A release has no previous version to return to. The canary watches only latency, not quality. Everyone assumes someone else approved the model.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Model registry, packaging and safe releases fits together

Part	What it does	What goes wrong
What a registry holds	A registry stores each model version with its run id, code and data versions, metrics, owner, and a stage such as staging, production or archived.	No named owner or approval for a model.
Package it like a service	Wrap the model in a container with pinned libraries, a clear input schema and a health endpoint.	Copying model files by hand.
Shadow release	In shadow mode the new model receives a copy of real traffic, but its answers are only logged, not used.	Canarying with only latency checks.
Canary and A/B	A canary sends a small share of traffic, say 1 to 5 percent, to the new model and watches errors, latency and quality metrics before widening.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Rollback	Keep the previous model loaded or one command away.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Moving a model through stages	The registry holds the version, the image digest and the previous version.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What a registry holds to Package it like a service / Package it like a service to Shadow release / Shadow release to Canary and A/B / Canary and A/B to Rollback / Rollback to Moving a model through stages

Incident: A rollback that took a day

> **Incident:** *A new model started giving poor results in production. The team wanted to go back, but the old model had been deleted from the server and its training environment no longer existed. Rebuilding it took most of a day.*

```
$ mlreg show churn --stage archived
version: 13  artifact: MISSING
$ mlreg rollback churn --to 13
error: artifact not found, rebuild required
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Keeping the previous model artifact and its image in the registry, and treating rollback as a stage change, turns an all-day rebuild into a minute-long switch.

Interview question

"How do you release a new model safely?" Register it with lineage, package it, run shadow traffic, then a small canary with quality and latency checks, promote in steps, and keep the previous version ready for an instant rollback.

Common mistakes

- Copying model files by hand
- Deleting old versions after release
- Canarying with only latency checks
- No named owner or approval for a model

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does shadow mode do?

Answer: Runs the new model on real traffic without using its answers

Why record the image digest in the registry?

Answer: So the exact tested artifact is the one that runs

What makes rollback fast?

Answer: The previous version stays available and switching is a stage change

When is an A/B test more suitable than a canary?

Answer: When you must measure a business outcome between groups

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Serving models: batch, online and streaming

15 minutes. Choose how predictions are produced and size the service for latency, throughput and cost.

There are several ways to turn a model into predictions, and the right one depends on how fast an answer is needed. Overnight scoring, a request-response API and a stream of events each have different costs and failure modes. This lesson helps you pick one, and shows what to measure so the service stays fast and affordable.

What you will learn

- Choose between batch, online and streaming serving
- Reason about latency, throughput and batching
- Scale and protect a model service
- Handle slow, large or failing model calls

Batch

Batch scoring runs the model over a large table on a schedule, then stores the results for other systems to read. It is cheap and simple, and it suits recommendations refreshed daily or risk scores for reports. The cost is staleness: the answer is as old as the last run.

Online

Online serving answers one request at a time through an API, usually in tens to hundreds of milliseconds. It suits fraud checks, search ranking and anything that depends on the latest event. It needs capacity for peaks, health checks, timeouts and a fallback when the model is down.

Streaming

Streaming scores events as they arrive from a queue. It sits between the two: near real-time, high volume and decoupled from the caller. You must plan for ordering, duplicates, backlog and what happens when the model is slower than the incoming rate.

Latency and batching

Model inference often runs much faster per item when several requests are grouped. Dynamic batching waits a few milliseconds to collect requests and then runs them together. It raises throughput but adds a little delay, so set a maximum wait that fits your latency budget.

Scale and protect

Autoscale on a signal that reflects load, such as queue depth or concurrent requests, not only CPU. GPU services start slowly because models are large, so keep a warm minimum. Set timeouts, limit concurrency, and return a safe default or an older cached answer when the model cannot respond.

MARGA by EknathaLabs marga.eknathalabs.com

A model service with sensible limits

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: churn-model}
spec:
  replicas: 3
  selector: {matchLabels: {app: churn-model}}
  template:
    metadata: {labels: {app: churn-model}}
    spec:
      containers:
      - name: server
        image: registry.example.com/churn@sha256:4f2a1c
        resources:
          requests: {cpu: "2", memory: 4Gi}
          limits: {memory: 6Gi}
        readinessProbe:
          httpGet: {path: /ready, port: 8080}
          initialDelaySeconds: 20
        env:
        - {name: MAX_BATCH_SIZE, value: "16"}
        - {name: MAX_BATCH_WAIT_MS, value: "8"}
        - {name: REQUEST_TIMEOUT_MS, value: "250"}
```

The readiness probe waits for the model to load. Batching is capped at eight milliseconds and a request timeout protects callers. Set these from your latency budget, not by guesswork.

How it fails

A model takes minutes to load, so new pods receive traffic too early. Batching waits too long and breaks the latency budget. A traffic spike overloads the service and callers pile up retries. No fallback exists when the model is down.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Serving models: batch, online and streaming fits together

Part	What it does	What goes wrong
Batch	Batch scoring runs the model over a large table on a schedule, then stores the results for other systems to read.	CPU-only autoscaling for a GPU model.
Online	Online serving answers one request at a time through an API, usually in tens to hundreds of milliseconds.	Online serving for something that needs no freshness.
Streaming	Streaming scores events as they arrive from a queue.	Serving traffic before the model has loaded.
Latency and batching	Model inference often runs much faster per item when several requests are grouped.	No timeout or fallback for the model call.
Scale and protect	Autoscale on a signal that reflects load, such as queue depth or concurrent requests, not only CPU.	Set timeouts, limit concurrency, and return a safe default or an older cached answer when the model cannot respond.
A model service with sensible limits	The readiness probe waits for the model to load.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Batch to Online / Online to Streaming / Streaming to Latency and batching / Latency and batching to Scale and protect / Scale and protect to A model service with sensible limits

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Retries that finished the job

> **Incident:** *A model service slowed down under load. Callers timed out and retried, which doubled the load and made it slower still, until the service stopped answering. The team had no concurrency limit and no fallback.*

```
$ kubectl top pod -l app=churn-model
NAME            CPU    MEMORY
churn-model-0   3980m  5.1Gi
$ curl -s -o /dev/null -w "%{http_code} %{time_total}\n" localhost:8080/predict
503 4.201
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration of a retry storm. A concurrency limit, a retry budget with backoff on callers, and a cheap fallback answer keep a slow model from taking down everything that depends on it.

Interview question

"Batch or online for a recommendation feature?" Ask how fresh the answer must be. If daily is fine, batch is cheaper and simpler. If it must react to the last click, use online with a fallback to the batch result.

Common mistakes

- Online serving for something that needs no freshness
- CPU-only autoscaling for a GPU model
- Serving traffic before the model has loaded
- No timeout or fallback for the model call

Check yourself

When is batch scoring the best fit?

Answer: When results can be a few hours old and volume is large

What does dynamic batching trade?

Answer: A little added delay for higher throughput

Why use a readiness probe for a model service?

Answer: The model may take time to load and should not get traffic before it is ready

What helps prevent a retry storm?

Answer: A concurrency limit, retry budget with backoff and a fallback

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Monitoring drift and retraining

15 minutes. Detect when data or the world has changed, measure model quality when labels arrive late, and retrain safely.

A model is trained on a snapshot of the past. The world keeps moving, so over time the data it sees and the patterns it learned drift apart. Monitoring tells you when. A retraining plan tells you what to do. This final lesson ties the earlier ones together into the loop that keeps a model useful.

What you will learn

- Tell data drift from concept drift
- Choose signals to monitor when labels arrive late
- Set alert thresholds that do not cry wolf
- Design a safe retraining trigger and process

Data drift and concept drift

Data drift means the inputs changed: more mobile users, a new product, a different season. Concept drift means the relationship between inputs and the right answer changed: customers behave differently for the same features. Data drift is easy to measure. Concept drift needs labels or a trusted proxy.

What to monitor

Watch input features (null rate, range, distribution against a training reference), predictions (average score, share in each class) and, when labels arrive, real quality such as precision, recall or error. Add the business metric the model serves. Group results by important segments.

When labels are late

Labels often arrive days or weeks later: a loan defaults months after approval. Until then, rely on leading signals: input drift, prediction drift and proxies. Plan a delayed quality report when labels land, and compare it with what the early signals said.

Thresholds and noise

Distribution tests will flag tiny, harmless shifts if you run them on large data. Pick a few features that matter, choose a drift measure with a meaningful scale, and alert on sustained shifts over a window rather than one-off blips. Tune using past incidents, so alerts match real harm.

Retraining

Retrain on a schedule, on a trigger such as drift or a quality drop, or both. Every retrain goes through the same pipeline and gate as a new model, including a comparison with production and a staged release. Automatic retraining without a gate is how a data problem turns into a bad release.

A drift alert rule

```
groups:
- name: model-drift
  rules:
  - alert: FeatureDriftHigh
    expr: model_feature_psi{model="churn",feature="days_since_login"} > 0.25
    for: 6h
    labels: {severity: ticket}
    annotations:
      summary: "Input drift on days_since_login for 6 hours"
      runbook: "Check upstream pipeline, compare with training reference"
  - alert: PredictionShare
    expr: abs(model_positive_rate{model="churn"} - 0.18) > 0.06
    for: 12h
    labels: {severity: ticket}
```

MARGA by EknathaLabs marga.eknathalabs.com

The first rule uses a population stability index with a six hour window. The second watches the share of positive predictions. Both open a ticket, not a page, because drift is slow and needs investigation, not a midnight wake-up.

How it fails

Alerts fire constantly on harmless changes, so people mute them. Nobody watches quality because labels are late. A retrain is triggered automatically with bad data. Drift is measured on features nobody cares about.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Monitoring drift and retraining fits together

Part	What it does	What goes wrong
Data drift and concept drift	Data drift means the inputs changed: more mobile users, a new product, a different season.	Check the Common mistakes list below this diagram for what usually goes wrong here.
What to monitor	Watch input features (null rate, range, distribution against a training reference), predictions (average score, share in each class) and, when labels arrive, real...	Ignoring segments that matter to the business.
When labels are late	Labels often arrive days or weeks later: a loan defaults months after approval.	Retraining automatically with no gate.
Thresholds and noise	Distribution tests will flag tiny, harmless shifts if you run them on large data.	Alerting on every tiny distribution change.
Retraining	Retrain on a schedule, on a trigger such as drift or a quality drop, or both.	Never comparing the model with a simple baseline.
A drift alert rule	The first rule uses a population stability index with a six hour window.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Data drift and concept drift to What to monitor / What to monitor to When labels are late / When labels are late to Thresholds and noise / Thresholds and noise to Retraining / Retraining to A drift alert rule

Incident: The model that aged quietly

> **Incident:** *A demand forecasting model was accurate at launch. A change in customer behaviour over several months made its forecasts steadily worse. No quality metric was reviewed, and the team found out when stock shortages appeared.*

```
$ python quality_report.py --model demand --window 90d
month   mae   baseline_mae
2026-01 41    44
2026-02 48    44
2026-03 63    44   ABOVE BASELINE
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Comparing error with a simple baseline every month, and alerting when the model is no longer better than that baseline, makes slow decay visible while it is still cheap to fix.

Interview question

"How do you know when to retrain?" Monitor input and prediction drift as early signals, measure real quality as labels arrive, compare with a simple baseline, and trigger a retrain through the normal gated pipeline when drift or quality crosses a limit.

Common mistakes

- Alerting on every tiny distribution change

MARGA by EknathaLabs
marga.eknathalabs.com

- Retraining automatically with no gate
- Never comparing the model with a simple baseline
- Ignoring segments that matter to the business

Check yourself

What is concept drift?

Answer: The relationship between inputs and the right answer has changed

What can you monitor when labels arrive late?

Answer: Input drift, prediction distribution and proxy metrics, then quality when labels land

Why should a retrain go through the same gate as a new model?

Answer: A retrain on bad data can produce a worse model

Why use a baseline in quality reports?

Answer: It shows whether the model still beats a simple approach

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Debugging ML systems in production

16 minutes. A bottom-up checklist for a model that is wrong, slow or silent in production: data first, then features, then the model, then serving.

When a model misbehaves, the model is rarely the first thing to blame. Work from the data upward and change one layer at a time, so that each check rules something out.

What you will learn

- Walk a bottom-up checklist: data, features, model, serving
- Tell training-serving skew from real drift using logged inputs
- Replay a bad prediction from stored inputs and the model version
- Write the regression test that stops the same fault returning

Start with what changed

List what changed in the last week: data sources, feature code, model version, serving image, traffic mix. Most production faults follow a change. If nothing changed, suspect slow drift or an upstream system that changed without telling you.

Check the data, then the features

Compare today's inputs with the training set: null rates, ranges, new category values, row counts. Then compare the feature values the service computed with the values the training pipeline would compute for the same record. A difference is skew, and skew is a code fault, not a model fault.

Replay the prediction

Log the model version, the input features and the output for a sample of requests. With those three you can replay one bad prediction offline, get the same answer, and then change one thing at a time to see which change fixes it.

Then the model, then serving

Only when data and features match do you look at the model: evaluate the served version on a fresh labelled slice and compare with its gate result. Last, check serving: wrong model version loaded, stale cache, a fallback silently answering instead of the model.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Debugging ML systems in production fits together

Part	What it does	What goes wrong
Start with what changed	List what changed in the last week: data sources, feature code, model version, serving image, traffic mix.	Not logging inputs, outputs and model version.
Check the data, then the features	Compare today's inputs with the training set: null rates, ranges, new category values, row counts.	Fixing it without adding a check that would have caught it.
Replay the prediction	Log the model version, the input features and the output for a sample of requests.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Then the model, then serving	Only when data and features match do you look at the model: evaluate the served version on a fresh labelled slice and compare with its gate result.	Retraining before checking the data.
Make it stay fixed	Add a data check at the pipeline boundary, a test that computes a feature the same way in training and serving, and an alert on the signal that would have caught this.	Check the Common mistakes list below this diagram for what usually goes wrong here.
A bottom-up check from the shell	The data check finds the null rate, the comparison shows serving computes the feature differently, and the replay reproduces the bad output.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Start with what changed to Check the data, then the features / Check the data, then the features to Replay the prediction / Replay the prediction to Then the model, then serving / Then the model, then serving to Make it stay fixed / Make it stay fixed to A bottom-up check from the shell

Predict before you look

Say what you expect each check to show before running it. A wrong guess teaches you where your picture of the system is wrong.

Make it stay fixed

Add a data check at the pipeline boundary, a test that computes a feature the same way in training and serving, and an alert on the signal that would have caught this. Write down the cause in one paragraph.

A bottom-up check from the shell

```
$ python check_inputs.py --window 48h --baseline training
feature days_since_login: 31% null (training: 0.2% null)
$ python compare_features.py --sample 500 --serving-vs-training
feature days_since_login: 29% of rows differ between serving and training
$ python replay.py --request-id 7f3a --model churn-v14
replayed output 0.12, logged output 0.12, expected 0.81
```

The data check finds the null rate, the comparison shows serving computes the feature differently, and the replay reproduces the bad output. The model is fine. The output is a simulated illustration.

How it fails

The team retrains the model first, which changes nothing because the fault is in a feature. Nobody logs inputs, so the bad prediction cannot be replayed. Several things are changed at once, so no one knows which fixed it.

Incident: The retrain that fixed nothing

> **Incident:** *Conversion from recommendations fell for two days. The team retrained the model twice with no improvement. A feature pipeline had started filling a column with nulls after an upstream schema change.*

```
$ python check_inputs.py --window 48h --baseline training
feature days_since_login: 31% null (training: 0.2% null)
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. A data check at the pipeline boundary would have caught the nulls the day they began, before any retraining was tried.

Interview question

"A model's results got worse in production. Where do you start?" Check what changed, then data, then features against training, replay a bad prediction from logs, and only then the model and serving. Change one thing at a time and add a test afterwards.

Common mistakes

- Retraining before checking the data
- Not logging inputs, outputs and model version
- Changing several layers at once
- Fixing it without adding a check that would have caught it

Check yourself

Why start with data and features rather than the model?

Answer: Most production faults come from changed inputs or feature code, and each check rules out a layer

What do you need logged to replay a bad prediction?

Answer: The model version, the input features and the output

What does a difference between served and training feature values show?

Answer: Training-serving skew, which is a code fault

Why change one layer at a time?

Answer: So you know which change fixed the problem

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Feature stores and data versioning

12 minutes. Keep training and serving features consistent with a feature store, and version datasets so any model can be reproduced.

Most ML bugs are data bugs. If a feature means one thing in the training notebook and another in the serving code, the model learns something the service cannot reproduce. Shared definitions and versioned data remove that gap.

What you will learn

- Training and serving skew and why it happens
- What a feature store provides
- Point-in-time correctness and leakage
- Versioning datasets and lineage

Skew

Training reads history from a warehouse with batch code. Serving computes the same value from live data with different code. Small differences, such as null handling, time zones or rounding, cause a model to behave worse than any offline test suggested.

One definition

```
# a feature defined once, used for offline and online
from datetime import timedelta
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64

customer = Entity(name="customer_id", join_keys=["customer_id"])
orders_stats = FeatureView(
    name="orders_stats",
    entities=[customer],
    ttl=timedelta(days=2),
    schema=[Field(name="orders_7d", dtype=Int64), Field(name="avg_basket", dtype=Float32)],
    source=FileSource(path="data/orders_stats.parquet", timestamp_field="event_ts"),
)
```

The **offline store** holds history for training, and the **online store** serves the latest values with low latency. A materialization job keeps them in sync.

Point in time

When building training rows, use only data that was available **at the time of each label**. Using a later value leaks the future into the model and inflates results. Feature stores provide point-in-time joins for this.

Version everything

Record the dataset version, feature definitions, code commit and parameters with every trained model. Tools such as DVC, lakehouse table versions or MLflow datasets let you reload exactly what a model saw.

```
dvc add data/train.parquet
git add data/train.parquet.dvc && git commit -m "train data v12"
mlflow runs describe --run-id "$RUN_ID"
```

Monitor **freshness** of online features. Stale features silently degrade predictions.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Feature stores and data versioning fits together

Part	What it does	What goes wrong
Skew	Training reads history from a warehouse with batch code.	Separate code for training and serving features.
One definition	The offline store holds history for training, and the online store serves the latest values with low latency.	Using future data in training rows.
Point in time	When building training rows, use only data that was available at the time of each label.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Version everything	Record the dataset version, feature definitions, code commit and parameters with every trained model.	Ignoring stale online features.

Flow: Skew to One definition / One definition to Point in time / Point in time to Version everything

Incident: Perfect offline, poor online

Incident: *A model's accuracy dropped after launch because of a feature mismatch.*

```
offline AUC: 0.91
online AUC first week: 0.74
feature avg_basket differs from offline value in 38% of requests
```

The service computed an average with a different window than the training job. Comparing logged online features with the offline values revealed it. The team moved both to one feature definition and added a daily skew check.

Common mistakes

- Separate code for training and serving features
- Using future data in training rows
- Unversioned datasets
- Ignoring stale online features

Check yourself

What is training-serving skew?

Answer: Features differ between training and serving

What is point-in-time correctness?

Answer: Using only data available at label time

What does the online store provide?

Answer: Low-latency latest feature values

Why version datasets?

Answer: To reproduce any model

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Model evaluation, A/B tests and shadow deployments

13 minutes. Judge a model with offline metrics, slices, shadow traffic and controlled online experiments, and decide when to ship or roll back.

Shipping a model is a change to a live system, so it deserves the same discipline as any release. The extra difficulty is that quality is statistical and can differ between groups of users.

What you will learn

- Choosing metrics that match the goal
- Slice analysis and fairness checks
- Shadow deployments
- A/B tests, guardrails and rollback

Metrics and slices

Pick a primary metric tied to the business goal and add guardrails such as latency, cost and error rate. Overall averages hide problems, so compute metrics by **slice**: new versus returning users, region, device, language and rare classes.

```
import pandas as pd
df = pd.read_parquet("eval_predictions.parquet")
by_slice = df.groupby("segment").apply(lambda g: (g.pred == g.label).mean())
print(by_slice.sort_values().head())
```

Shadow deployment

In **shadow mode** the new model receives a copy of live requests, but its output is not used. Compare outputs, latency and errors with the current model. This catches crashes, performance problems and surprising predictions without risking users.

Online experiments

```
experiment: ranker-v7
split: {control: 90, treatment: 10}
primary_metric: purchase_rate
guardrails: {p95_latency_ms: "<= 250", error_rate: "<= 0.5%"}
min_duration_days: 14
stop_rules: {guardrail_breach: abort}
```

Decide the duration and sample size before starting, so you do not stop when the numbers look good. Random assignment must be stable per user. Watch guardrails continuously and abort on a breach. After a win, ramp up gradually and keep the old model available for quick rollback.

Record

Store the evaluation report with the model version, data version and decision. This helps future reviews and audits.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Model evaluation, A/B tests and shadow deployments fits together

Part	What it does	What goes wrong
Metrics and slices	Pick a primary metric tied to the business goal and add guardrails such as latency, cost and error rate.	Judging only by overall averages.
Shadow deployment	In shadow mode the new model receives a copy of live requests, but its output is not used.	This catches crashes, performance problems and surprising predictions without risking users.
Online experiments	Decide the duration and sample size before starting, so you do not stop when the numbers look good.	Stopping an experiment when it first looks good.
Record	Store the evaluation report with the model version, data version and decision.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Metrics and slices to Shadow deployment / Shadow deployment to Online experiments / Online experiments to Record

Incident: Better on average, worse for new users

Incident: *A model improved overall metrics but hurt a user segment.*

```
overall conversion: +1.8%
new users: -6.2%
returning users: +3.1%
```

The model relied on purchase history that new users did not have. The overall average hid the drop because new users were a minority. The team added a slice report to every evaluation and a fallback model for users with no history.

Common mistakes

- Judging only by overall averages
- Stopping an experiment when it first looks good
- No guardrail metrics
- No way to roll back quickly

Check yourself

What does shadow mode do?

Answer: Runs a model on live traffic without using output

Why analyze slices?

Answer: Averages can hide harm to groups

What is a guardrail metric?

Answer: A limit that must not be breached

When decide the sample size?

Answer: Before the experiment

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. ML pipelines and workflow orchestration

12 minutes. Run training as a repeatable pipeline with steps, caching, retries, parameters and scheduled retraining on an orchestrator.

A pipeline makes every step explicit: where data comes from, how it is checked, how the model is trained and what must be true before it is released. That is the foundation for continuous training.

What you will learn

- Breaking work into pipeline steps
- Passing data and artifacts between steps
- Retries, caching and resource requests
- Scheduling and triggering retraining

Steps

A typical pipeline: **ingest** data, **validate** it, **build features**, **train**, **evaluate** against a threshold, **register** the model and optionally **deploy**. Each step is a container or function with clear inputs and outputs.

```
from airflow.decorators import dag, task
from datetime import datetime

@dag(schedule="0 3 * * 1", start_date=datetime(2026, 1, 1), catchup=False)
def retrain_churn():
    @task(retries=2)
    def validate(ds=None):
        return run_data_checks(ds)           # returns a dataset version

    @task
    def train(dataset_version):
        return train_model(dataset_version)  # returns a model run id

    @task
    def evaluate(run_id):
        score = evaluate_model(run_id)
        if score < 0.82:
            raise ValueError("model below threshold")
        return run_id

    @task
    def register(run_id):
        register_model(run_id, stage="staging")

    register(evaluate(train(validate())))

retrain_churn()
```

Data between steps

Pass **references**, such as dataset versions and object storage paths, not large data through the orchestrator. Store artifacts in object storage or an artifact store with versioning.

Reliability

Set retries with backoff for transient failures, but not for logic errors. Cache steps when inputs and code have not changed. Set resource requests, for example GPUs for training only. Make each step **idempotent**, so rerunning does not duplicate or corrupt results.

Triggers

Run on a schedule, when new data arrives or when drift monitoring requests it. Require a successful evaluation gate before anything is registered, and send alerts on failed runs.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How ML pipelines and workflow orchestration fits together

Part	What it does	What goes wrong
Steps	A typical pipeline: ingest data, validate it, build features, train, evaluate against a threshold, register the model and optionally deploy.	Passing large data through the orchestrator.
Data between steps	Pass references, such as dataset versions and object storage paths, not large data through the orchestrator.	Steps that are not idempotent.
Reliability	Set retries with backoff for transient failures, but not for logic errors.	Notebooks as the only process.
Triggers	Run on a schedule, when new data arrives or when drift monitoring requests it.	No evaluation gate before registration.

Flow: Steps to Data between steps / Data between steps to Reliability / Reliability to Triggers

Incident: The notebook that nobody could rerun

Incident: *A model could not be retrained because the process lived in a notebook.*

```
FileNotFoundError: /Users/amir/data/clean_v3.csv
notebook cell 14 depends on cell 31 being run first
retraining blocked for 3 weeks
```

The author had left, and the notebook referenced a local file and an undocumented order of cells. The team rebuilt the process as a pipeline with versioned data, explicit steps and tests, and retrained in an afternoon.

Common mistakes

- Notebooks as the only process
- Passing large data through the orchestrator
- Steps that are not idempotent
- No evaluation gate before registration

Check yourself

What should pipeline steps pass between each other?

Answer: References to versioned artifacts

What does an evaluation gate do?

Answer: Blocks models below a threshold

Why make steps idempotent?

Answer: Reruns do not corrupt results

What can trigger retraining?

Answer: Schedule, new data or drift

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. LLMOps: versioning prompts, models and evaluations

12 minutes. Run language model applications like software: version prompts and models, test with evaluation sets, monitor cost and quality and release carefully.

Classic MLOps manages models you train. LLMOps manages applications built on models you call, where behavior is changed mostly through prompts, retrieval and model selection. The discipline is the same: version, test, release, observe.

What you will learn

- Prompts as versioned artifacts
- Pinning models and planning upgrades
- Evaluation suites in CI
- Monitoring cost, latency and quality

Prompts in version control

```
# prompts/ticket_summary/v3.yaml
name: ticket_summary
version: 3
model: claude-sonnet-4-5-20250929
temperature: 0.2
system: |
  You summarize support tickets for engineers. Be factual and brief.
template: |
  Ticket:
  {{ticket_text}}
  Write a summary of at most 3 sentences and list the product area.
```

Store prompts with their model settings, review changes in pull requests and tag releases. Avoid editing prompts only in a vendor console where history is lost.

Pin and upgrade

Use specific model versions instead of moving aliases. When a new model appears, run the evaluation suite, compare quality and cost and roll out gradually.

Evaluate in CI

```
python evals/run.py --prompt prompts/ticket_summary/v3.yaml --cases evals/tickets.jsonl --out resu \
    lts.json
python evals/check.py results.json --min-score 0.85 --max-cost-per-1k 4.00
```

Cases come from real examples. Combine exact checks (valid JSON, required fields), rule checks (no personal data) and graded checks for quality. Fail the pipeline on regression.

Observe

Record per request: prompt version, model version, input and output tokens, latency, cost, tool calls and user feedback, with sensitive data redacted. Alert on cost per request, error rate and format failures. Add failing real cases to the evaluation set, so the suite improves with experience.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How LLMOps: versioning prompts, models and evaluations fits together

Part	What it does	What goes wrong
Prompts in version control	Store prompts with their model settings, review changes in pull requests and tag releases.	Prompts edited in consoles without history.
Pin and upgrade	Use specific model versions instead of moving aliases.	Using moving model aliases in production.
Evaluate in CI	Cases come from real examples. Combine exact checks (valid JSON, required fields), rule checks (no personal data) and graded checks for quality.	Not tracking cost per request.
Observe	Record per request: prompt version, model version, input and output tokens, latency, cost, tool calls and user feedback, with sensitive data redacted.	No evaluation set.

Flow: Prompts in version control to Pin and upgrade / Pin and upgrade to Evaluate in CI / Evaluate in CI to Observe

Incident: The alias that changed

Incident: *A model alias moved to a new version and broke JSON parsing.*

```
model: provider-latest
output now starts with "Sure, here is the JSON:"
parser error rate: 0% to 17%
```

The team used an alias for convenience, and the provider updated it. The new version added a polite preface before the JSON. They pinned an exact model version, added a format check to evaluations and scheduled upgrades with testing.

Common mistakes

- Using moving model aliases in production
- Prompts edited in consoles without history
- No evaluation set
- Not tracking cost per request

Check yourself

Why pin model versions?

Answer: Behavior should not change unexpectedly

Where store prompts?

Answer: Version control with review

What gates a prompt change?

Answer: Evaluation results

What should be logged per request?

Answer: Versions, tokens, latency and cost

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: MLOps

Checks, records and release steps that keep machine learning models reliable in production.

What to monitor

Layer	Signals	Why
Service	latency, errors, saturation	The usual health of the API
Inputs	null rate, range, new categories, distribution vs training	Catches broken or changed data
Predictions	average score, share per class	Early sign of drift
Quality	accuracy or error when labels arrive, vs a baseline	The real measure
Business	conversion, loss, complaints	What the model is for

Warning: *A healthy service does not mean a good model. Models fail quietly.*

Record for every training run

```
code version      a94f3c2
data version      churn-2026-03-01
parameters        learning_rate=0.05 max_depth=6
seed              42
environment       image digest, library versions, hardware
metrics           overall and per important group
artifacts         model file, evaluation report
```

Data checks before training

```
checks:
  - row_count: {min: 90000, max: 130000}
  - freshness: {column: event_date, max_age_hours: 30}
  - not_null: [customer_id, event_date]
  - range: {column: days_since_login, min: 0, max: 3650}
  - allowed_values: {column: plan_tier, values: [free, basic, pro, team]}
on_fail: stop_pipeline_and_page
```

Release steps

1. Register the model with run id, data version and metrics.
2. Package it in a pinned image and record the digest.
3. Gate: compare with production on a fixed set, including key groups.
4. Shadow on real traffic, then a small canary with quality checks.
5. Promote in steps. Keep the previous version ready.

Serving choices

Mode	Use when	Watch
Batch	results can be hours old	staleness, job failures
Online	the latest event matters	latency, timeouts, fallback
Streaming	high volume, near real time	backlog, duplicates

Drift alert example

```
- alert: FeatureDriftHigh
  expr: model_feature_psi{model="churn",feature="days_since_login"} > 0.25
  for: 6h
  labels: {severity: ticket}
```

Why it matters: *Drift is slow. A ticket with a window is better than a midnight page.*

MARGA by EknathaLabs
marga.eknathalabs.com

Debug a model bottom-up

1. What changed in the last week: data, feature code, model, image, traffic.
2. Inputs against training: null rate, range, new categories.
3. Served features against training features for the same record.
4. Replay a logged request with the same model version.
5. Only then the model, and last serving.

Warning: *Do not retrain before checking the data. A retrain on broken inputs repeats the fault.*

Warning: *Without logged inputs, output and model version, a bad prediction cannot be replayed.*

Warning: *Do not let a release pass on one overall metric. Check the groups that matter, such as new customers.*

Warning: *Do not retrain automatically with no gate. A retrain on bad data can produce a worse model.*

Commands for a model check

```
$ python check_inputs.py --window 48h --baseline training
```

```
$ python compare_features.py --sample 500 --serving-vs-training
```

```
$ python replay.py --request-id 7f3a --model churn-v14
```

MARGA by EknathaLabs
marga.eknathalabs.com